Links:- https://docs.mistral.ai/api

https://platform.openai.com/docs/api-reference/chat/create

1. What happens inside an LLM (quick recap)

A Large Language Model (LLM) works by predicting the **next token** again and again. At every step, it looks at the tokens so far and assigns probabilities to all possible next tokens. The token with the highest probability is not always chosen automatically; the final choice depends on how the model is configured.

Modern LLMs use the **transformer architecture** with **self-attention**. Self-attention helps the model understand context, meaning, and relationships between words across the entire input, not just nearby words.

The key idea is simple: the model does not "think" in sentences. It continuously predicts the next token based on probabilities.

2. Prompt is important, but not the only thing

A **prompt** is the input that guides the model. It strongly influences *which tokens* are considered and *how probabilities are distributed* among them.

For example, if the prompt is “What is photosynthesis?”, the model gives higher probability to tokens like *photosynthesis*, *plants*, *sunlight*, and *process*.

If the prompt is “Write a horror story”, the probability mass shifts toward tokens like *dark*, *whispers*, *shadows*, and *blood*.

However, the prompt alone does **not** decide the final output. This is the main catch.

3. The second major factor: Model Configuration

Model configuration is the set of **settings** that control how the model behaves *while generating text*. These settings do not change what the model has learned. They only control how the learned knowledge is used during generation.

Think of it like this: learning is already done. Configuration decides *how boldly or cautiously* the model speaks, *how random or deterministic* it is, and *how much repetition* it allows.

Prompt + Model Configuration together decide the final output.

4. What model configuration does NOT do

Model configuration does **not** retrain the model. The internal knowledge and weights remain the same.

It does **not** add new facts or remove existing knowledge.

It only controls the **selection process** after probabilities are already calculated.

5. A simple probability example

After reading a prompt, the model may internally reach a probability distribution like this for the next token:

A → 0.40
B → 0.39
C → 0.21

Now the model must choose **one** token. This choice is controlled by model configuration. Depending on the settings, it may always pick A, or sometimes pick B or C.

6. Sampling: how tokens are finally chosen

Sampling parameters decide **how one token is selected** from the probability distribution.

Without sampling, the model would always pick the highest-probability token, making outputs repetitive and boring.

Sampling introduces controlled randomness.

7. Temperature

Temperature controls how *random* or *confident* the model is.

At low temperature, the model strongly prefers high-probability tokens. Outputs become more factual, predictable, and stable.

At high temperature, probabilities are flattened. Lower-probability tokens get a better chance. Outputs become more creative but also more risky.

Temperature values are usually between **0 and 1**, but can go slightly above 1 in some systems.

Very low temperature feels like answering an exam. Very high temperature feels like brainstorming ideas.

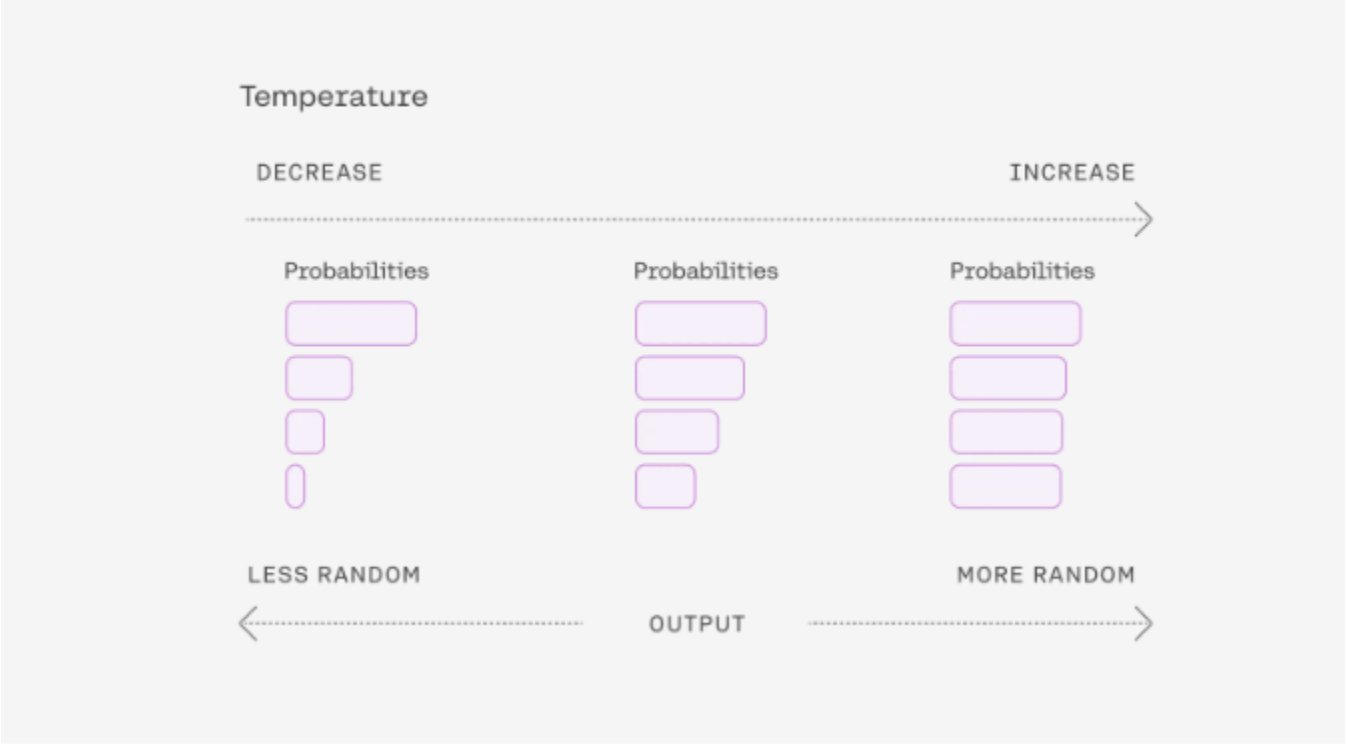
Temperature mathematically reshapes the probability distribution.

At temperature = 1, probabilities remain unchanged.

At temperature < 1, the distribution becomes sharper. High-probability tokens become even more dominant, and low-probability tokens almost disappear.

At temperature > 1, the distribution flattens. Differences between probabilities shrink, allowing more variation.

This is why temperature is often described as a **creativity dial**, but in reality it is a **confidence dial**. Lower temperature means the model is more confident in its top choice. Higher temperature means it is more exploratory.



8. Top-K sampling

Top-K limits the model to consider only the **K most probable tokens**.

If K = 50, the model ignores all tokens except the top 50 by probability. The final token is sampled only from this smaller set.

Lower K makes output safer and more focused. Higher K allows more variety.

Top-K acts like saying: “Only choose from the most reasonable options.”

9. Top-P (nucleus sampling)

Top-P does not fix the number of tokens. Instead, it selects the **smallest set of tokens whose combined probability is P**.

If Top-P = 0.9, the model keeps adding tokens until their total probability reaches 90%, and samples only from that group.

Top-P adapts automatically. Sometimes it may include 5 tokens, sometimes 50, depending on the distribution.

This often feels more natural than Top-K.

10. Using Top-K and Top-P together

Top-K and Top-P are often used together.

Top-K limits extreme outliers. Top-P adapts to the confidence of the model.

Many practitioners recommend keeping one stable and tuning the other, instead of aggressively tuning both.

11. Repetition Penalty

Repetition penalty discourages the model from repeating the same words or phrases again and again.

A penalty value of 1 means no penalty. Higher values reduce the probability of tokens that have already appeared.

This is useful to prevent loops and boring repetition, especially in long outputs.

12. Max tokens and stop sequences

Max tokens decide how long the output can be. Once the limit is reached, generation stops even if the sentence is incomplete.

Stop sequences tell the model when to stop early if a specific pattern appears. This is useful in structured outputs like code or Q&A formats.

13. Same model, different behavior

Using the same model with different configurations can feel like using different models.

One configuration may feel calm, concise, and factual. Another may feel creative, verbose, and exploratory.

This is why configuration is sometimes described as choosing between **Eco mode**, **Comfort mode**, and **Sport mode**.

14. Why prompt engineering alone is not enough

Prompt engineering shapes *what the model talks about*.

Model configuration shapes *how the model talks*.

Ignoring configuration leads to confusion when the same prompt gives different outputs across tools or runs.

15. Big picture summary

The prompt influences token selection and probability assignment.

The model configuration influences how one token is chosen from those probabilities.

Learning stays fixed. Behavior changes.

That is why prompts and model configuration must be understood together, not separately.

16. Going deeper: how probability is actually used during generation

When the model processes a prompt, it does not immediately generate words. Internally, it converts text into tokens and produces a **probability distribution** over the entire vocabulary for the next token. This distribution always sums to 1.

At this stage, the model has already done its "thinking". All knowledge, context understanding, and reasoning are reflected inside this probability distribution. What comes next is purely a **selection problem**.

This is where model configuration becomes critical. Without configuration, the model would always pick the highest-probability token (greedy decoding). That would make outputs extremely repetitive and predictable.